

CPT102 Data Structures: Lecture 1 : Introduction to Algorithms

Thomas Selig

XJTLU

March 8, 2026

- Learning Mall has link to BOX folder for recordings.
 - Please remind me when the link expires.
- TA recruitment (almost) complete.
 - Contact information in handbook.
 - Help-session booking system in Learning Mall.
- Exercise sheet on Learning Mall.
 - Covers Weeks 1 and 2.
 - Doesn't contribute to your grade, but should be helpful in preparing for assessments and exam.
 - Ask for help from teaching team (module leaders/TAs) if needed.

Content of today's lecture

Recall that last week's lecture introduced the first main strand of the course: **Data Structures**

Today's lecture introduces the second strand of the course: **Algorithms**

- Definition and examples.
- Representing algorithms using pseudo-code and flowcharts.
- Algorithms analysis: complexity and the Big-Oh notation.

Lecture learning outcomes

By the end of the lecture, you should be able to:

- A. Explain the concept of an algorithm.
- B. Utilise pseudo-code and flowcharts to represent simple algorithms.
- C. Analyse the complexity of simple iterative algorithms using the Big-Oh notation.

Algorithms: definition

Definition

An **algorithm** is a finite sequence of precise and well-defined instructions to solve a specified problem.

Let's break this definition down.

- The instructions are “precise and well-defined”: must be unambiguous, easy to follow.
 - We refer to the instructions as *steps* of the algorithm.
- The instructions form a “finite sequence”:
 - “sequence” $\Rightarrow$ the steps must be performed in the correct order;
 - “finite” $\Rightarrow$ the algorithm must have a stopping point; we say that the algorithm *terminates*.
- The goal of an algorithm is “to solve a specified problem”; when we apply the steps in the correct order, we get a solution to the problem.
 - The solution is the *output* of the algorithm.
 - An algorithm also (usually) has an *input*.

Activity 1: Algorithm examples

- Based on prior definition and comments, propose examples of algorithms.
 - These do not have to be computer-related.
- Specify what the input(s) and output(s) are.
- Describe the steps of the algorithm.
 - No need to write (pseudo-)code, plain English is sufficient.
- Answer in the Etherpad here.

Algorithm example from last week

Algorithm that checks if an expression is well-balanced. The expression is the *input* of the algorithm.

- ❶ Initialise an empty stack.
 - ❷ Read the expression from left-to-right:
 - If we encounter an opening delimiter, push it to the stack.
 - If we encounter a closing delimiter, pop the stack; if stack is empty, output FALSE.
 - If the popped (opening) delimiter matches the current closing delimiter, we have found a matching pair, and continue.
 - Otherwise, output FALSE.
 - ❸ Once we have finished reading the entire expression:
 - If the stack still has elements, output FALSE.
 - Otherwise, if the stack is empty, output TRUE.
- Outputs TRUE if and only if the expression is well-balanced.
 - Described in natural language (plain English).
 - N.B.: The stack is *not* part of the input.

Algorithm example?

Is the following an algorithm? (quick class poll)

Algorithm?

Make one billion dollars.

Representation of algorithms

How can we represent or write algorithms?

- In code? But then it's language-dependent.
- In plain English? But then some instructions may be unclear or imprecise.

In most of this course, we will represent our algorithms using:

- pseudocode;
- flowcharts.

Definition

Pseudocode is a notation system used to describe the steps in an algorithm using a mixture of natural language and high-level programming constructs.

- Intended to be read by humans, not computers.
- More structured than plain English.
- Easy to convert into actual computer code, only need to translate line-by-line.
- Notation system is not universal, but should be easily understood by any person with programming experience.

Why pseudocode?

Allows for language-independence representation of algorithms using a notation that is sufficiently precise to easily convert to actual code.

Pseudocode example: simple

Algorithm 1: Linear array search

Input: An array of integers `arr`; an integer to search `val`

Output: The index of first occurrence of `val` in `arr` if it exists, -1 otherwise

```
 $n \leftarrow \text{len}(\text{arr});$   
for  $i \leftarrow 0$  to  $n - 1$  do  
    if arr[i] = val then  
        return  $i$ ;  
return  $-1$ ;
```

- Here we explicitly describe the Input/Output, but we can also just write this as a **procedure** with parameters and returns (next slide).
- Note that we use “ $\leftarrow$ ” for assignment, and “=” to test equality. You can use “=” or “:=” instead for assignment, and “==” to test equality, but be consistent!
- Vertical bars are optional, but helpful to clearly indicate blocks.

Pseudocode example: more complex

Algorithm 2: Checking if expression is well-balanced

Procedure ISWELLBALANCED(expr)

Input: a String expr

$S \leftarrow \text{Stack}();$

▷ S is an empty Stack

$i \leftarrow 0;$

while $i < \text{len}(\text{expr})$ **do**

$c \leftarrow \text{expr}[i];$

▷ c is the i -th character of expr

if c is an opening delimiter **then**

$S.\text{push}(c);$

else if c is a closing delimiter **then**

if $S.\text{isEmpty}()$ **then**

return false;

else

$c' \leftarrow S.\text{pop}();$

if c and c' do not match **then**

return false;

$i \leftarrow i + 1;$

return $S.\text{isEmpty}();$

Pseudocode guidelines

- Mimic good code and good English.
 - Use descriptive variable names;
 - Include comments where useful;
 - English should be comprehensible (full sentences not needed).
- Ignore unnecessary details, omit information that is clear from context.
 - E.g., types of variables are often obvious.
- Ultimate goals of pseudocode:
 - Easy to understand;
 - Easy to convert into actual code;
 - Easy to analyse the underlying algorithm.

Activity 2: pseudocode reading

Algorithm 3: The mystery algorithm

Procedure MYSTERYALGORITHM(x, y)

Input: two positive integers x, y

$r \leftarrow x, q \leftarrow 0;$

while $r \geq y$ **do**

$r \leftarrow r - y;$
 $q \leftarrow q + 1;$

return $(q, r);$

What does the above algorithm output?

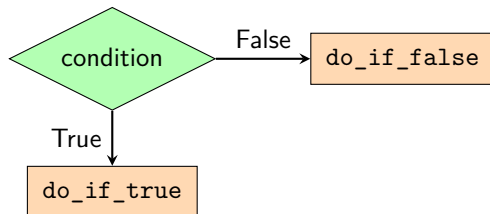
- Answer in the classroom.

Definition

A **flowchart** is a notation system used to represent processes in visually appealing, easy-to-follow, form.

- A flowchart represents the *flow* of a process.
 - Allows us to easily follow the process step-by-step.
- We will use them to represent algorithms.
- Flowchart forms are standard.

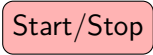
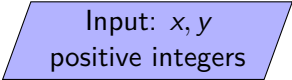
Flowchart: decision and process blocks



- The diamond  is a *decision block*.
 - Used for both conditional instructions and loops.
 - Limited to if/else (no “else if”) and while.
 - Sometimes written as a question “condition?”, in which case the outgoing arrows are labelled “Yes/No” rather than “True/False”.
- The rectangles  are *process blocks*.
 - Used for instructions/statements.
- Conventionally, the “do if true” arrow points downwards, while the “do if false” arrow points to the side.

Flowchart: other blocks

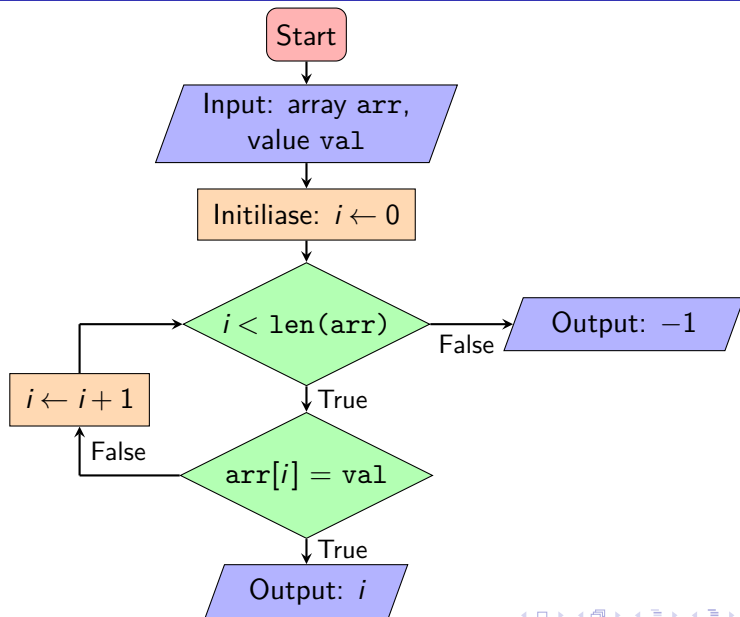
Two other types of block:

- Beginning and end blocks:  (rounded rectangle)
- Input and output blocks:  (parallelogram)

Remarks:

- Colours in these slides are a personal choice, shapes are standard.
- Instructions can be pseudocode, plain English, or a mixture of both.
- Descriptive variable names are still important.

Flowchart example: linear search



Summary: algorithms

- Definition: a finite sequence of precise, well-defined, concise steps to solve a specific problem.
- Some examples:
 - Non-computing: cookbook recipe, furniture assembly manual;
 - Computing: linear search, checking well-balanced.
- Representation through pseudocode and flowcharts.

Question

What makes a *good* algorithm?

“Good” algorithms

Question

What makes a *good* algorithm?

- Simple (easy to express and understand)?
- Fast (terminates quickly)?
- Memory-efficient (optimises memory usage)?
- All of the above?

Question

How to calculate the speed of an algorithm?

Towards the notion of complexity

How fast an algorithm terminates depends on many things:

- The language it is implemented in;
- The computer it is executed on (and its state, e.g. how many other tasks are running at the same time);
- The algorithm's input;
- ...

In general, measuring the amount of *physical time* an algorithm takes to run is impractical, and too dependent on other factors.

Instead, we measure how many *operations* the algorithm needs to perform.

- Known as the (time) **complexity** of the algorithm.

Time complexity: definition

Definition

The **time complexity** of an algorithm measures the amount of *elementary operations* that need to be performed in an execution of the algorithm.

Elementary operations? Essentially all basic operations that can be performed by a computer program, including:

- Assignments;
- Comparisons (for conditional statements and loops);
- Arithmetic operations (add, subtract, multiply, divide, modulo);
- Returns;
- And so on.

This model is equivalent to measuring the (physical) time taken by an algorithm in its execution, assuming that every elementary operation is performed in exactly one unit of time.

Time complexity: dependence on input

In the previous slide definition, the time complexity of an algorithm depends on its input. For example, consider `LINEARSEARCH`:

- If `arr[0] = val`, we perform only four operations:
 - $n \leftarrow \text{len}(\text{arr})$ (assignment);
 - $i \leftarrow 0$ (assignment);
 - check `arr[0] = val` (comparison);
 - **return** 0 (return);
- If `val` is not in the array, we perform 2 (assignments) + n (comparisons) + 1 (return) = $n + 2$ operations.
 - In this case, it depends on the *size* of the input.

Actually not quite correct: what's missing?

Questions

How bad can it be ($\equiv$ what's the *worst-case*)? What is the complexity for a "typical" input ($\equiv$ what's the *average-case*)?

Worst- and average-case time complexities

Definition

The **worst-case time complexity** of an algorithm is the function $T_{\text{worst}} : \mathbb{N} \rightarrow \mathbb{N}$ such that, for all $N \in \mathbb{N}$, $T_{\text{worst}}(N)$ is the maximal time complexity of the algorithm over all inputs of size N .

- Often just denoted $T(N)$.
- Gives a *theoretical guarantee* that, no matter what the input is, your algorithm will perform no worse than $T(N)$.
- What does “input of size N ” mean? Usually clear from context!
 - If the input is a collection, the size is just the number of elements in the collection (size of the collection).
 - If the input is a positive integer, the size is just the integer.
 - Sometimes we have two inputs with different sizes, and will write $T(M, N)$.

Example: linear search

Algorithm 4: linear search

```
1 Procedure LINEARSEARCH(arr, val)
2    $n \leftarrow \text{len}(\text{arr}), i \leftarrow 0$ ;
3   while  $i < n$  do
4     if  $\text{arr}[i] = \text{val}$  then
5       return  $i$ ;
6      $i++$ ;
7 return  $-1$ ;
```

Written as a while loop to make clear the increment and loop checks (missing from earlier calculation).

Worst-case analysis: linear search (1)

Here, the input size N is the size/length of the array.

- Why do we not consider the value searched for?

Case 1: The array contains the requested value, and returns $k \in [0, N - 1]$.

- Line 2: 2 assignments;
- For i from 0 to $k - 1$, we execute: check $i < n$ (Line 3), check $\text{arr}[i] = \text{val}$ (Line 4), Line 5 not executed, increment $i++$ (Line 6); $\text{total} = 3k$
- For $i = k$, we execute: check $i < n$ (Line 3), check $\text{arr}[i] = \text{val}$ (Line 4), **return** i (Line 5); $\text{total} = 3$

In total, we have $\tilde{T}(k) = 3k + 5$ elementary operations, where $k \leq N - 1$ is the index of first occurrence of val in arr .

Worst-case analysis: linear search (2)

Case 2: The array does not contain the requested value, and returns -1 . Analysis is very similar.

- Line 2: 2 assignments;
- For i from 0 to $N - 1$, we execute: check $i < n$ (Line 3), check $\text{arr}[i] = \text{val}$ (Line 4), Line 5 not executed, increment $i++$ (Line 6); $\text{total} = 3N$
- When $i = N$, we check that $i < N$ is **false** to exit loop (Line 3), then return -1 , giving 2 operations.

In total, we have $\hat{T}(N) = 2 + 3N + 2 = 3N + 4$ elementary operations.

Worst-case analysis: linear search (3)

- In summary, we execute:

Case 1: $\tilde{T}(k) = 3k + 5$ operations when the algorithm returns $k \in [0, N - 1]$.

Case 2: $\hat{T}(N) = 3N + 4$ operations when the algorithm returns -1 .

- Recall that we are looking for the *worst-case* time complexity, i.e. the maximal time complexity over all inputs.
- Note that for all $k \in [0, N - 1]$, we have:
 $\tilde{T}(k) \leq \tilde{T}(N - 1) = 3N + 2 < 3N + 4 = \hat{T}(N)$. As such, we get:

$$T(N) = \max \left\{ \max \left\{ \tilde{T}(k); k \in [0, N - 1] \right\}, \hat{T}(N) \right\} = 3N + 4.$$

Average-case time complexity

Definition

The **average-case time complexity** of an algorithm is the function $T_{\text{avg}} : \mathbb{N} \rightarrow \mathbb{N}$ such that, for all $N \in \mathbb{N}$, $T_{\text{avg}}(N)$ is the expected time complexity of the algorithm when given a *random* input of size N .

- Estimates the time complexity for a “typical” input, rather than in the worst-case scenario.
- What does “random input of size N ” mean? Unfortunately, not always obvious.
 - The complexity obtained can depend on the chosen distribution for the input.
- Almost always more complicated to calculate than worst-case.

Average-case analysis: linear search

Model: Entries in our input array are i.i.d. random integers, each with a fixed probability p of being equal to the value searched for.

- Let us denote $p_k := \mathbb{P}(\text{LINEARSEARCH}(\text{arr}, \text{val}) = k)$ the probability that the `LINEARSEARCH` algorithm returns k , for $k \in [0, N-1]$, and p_{FAIL} the probability that it returns -1 (searched value not found).
- Then we have: $T_{\text{avg}}(N) = \sum_{k=0}^{N-1} p_k \cdot \tilde{T}(k) + p_{\text{FAIL}} \cdot \hat{T}(N)$.
- Moreover, in our model, $(p_k)_{0 \leq k \leq N-1}$ is just a truncated geometric distribution, i.e.: $p_k = (1-p)^k \cdot p$ for $k \in [0, N-1]$.
- Finally, we have $p_{\text{FAIL}} = (1-p)^N$.

Bringing it all together, and using previous results, we get:

$$T_{\text{avg}}(N) = p \cdot \sum_{k=0}^{N-1} (1-p)^k (3k+5) + (1-p)^N (3N+4)$$

Towards Big-Oh notation

- This all seems rather complicated..
- In fact, we assumed that the increment operation $i++$ was a single operation.
 - Not true in all languages, e.g. Python would write: $i = i + 1$ which is 2 operations.
 - Also not quite true: a lot of Python compilers optimise this.
- But we don't want our analyses to be language-dependent, or (worse) distribution-dependent!!
- Our model is quite crude: not all elementary operations take the same time.
- Instead of exact calculations, algorithm analysis provides *estimates* of the complexity, using something known as “Big-Oh notation”.

Big-Oh notation: definition, basics, examples

Definition

Let $f, g: \mathbb{N} \rightarrow \mathbb{R}$ be two functions from the set of non-negative integers to the set of (strictly) positive real numbers. We say that “ $f(N)$ is Big-Oh of $g(N)$ ” and write $f(N) = O(g(N))$, if there exists some constant $C > 0$ such that $f(N) \leq C \cdot g(N)$ for all $N \in \mathbb{N}$.

Some basic properties.

- We always have $f(N) = O(f(N))$.
- For any constants $a, b > 0$, we have $a \cdot f(N) + b = O(f(N))$. In words, we can “throw away” additive and multiplicative constants.
- If $f(N) = O(g(N))$ and $g(N) = O(h(N))$, then $f(N) = O(h(N))$.

Examples:

- If $f(N) \leq a \cdot N + b$, then $f(N) = O(N)$.
- More generally, if $f(N) \leq P(N)$ for some polynomial function P with degree k , then $f(N) = O(N^k)$.

Quiz-time!

Consider the function $f(N) = 2N^2 + 6N \cdot \log(N) - \sqrt{N} + 3$. Which of the following statements are true?

- a $f(N) = O(N)$
- b $f(N) = O(N^2)$
- c $f(N) = O(N \cdot \log(N))$
- d $f(N) = O(N^2 - N)$
- e $f(N) = O(N^3)$

Which of these estimates is the most useful?

Complexity estimates

Objective

For a complexity $T(N)$, look for the simplest function $g(N)$ such that $T(N) = O(g(N))$, and where g is a “close estimate” for $T(N)$.

Typical scales, in increasing order.

- Constant: $g(N) = 1$.
- Logarithmic: $g(N) = \log(N)$.
- Sub-linear: $g(N) = N^a$ for some $0 < a < 1$.
- Linear: $g(N) = N$.
- Log-linear: $g(N) = N \cdot \log(N)$.
- Quadratic: $g(N) = N^2$.
- Polynomial: $g(N) = N^k$ (usually $N > 1$).
- Exponential: $g(N) = a^N$ for some $a > 1$.

Estimating complexities with Big-Oh notation is usually referred to as **asymptotic analysis** of algorithms.

Asymptotic analysis: linear search

Worst-case:

- Recall: $T_{\text{worst}}(N) = 3N + 4$. Easy! We have $T_{\text{worst}}(N) = O(N)$, so complexity is linear.
- Unsurprising: intuitively, worst-case is where we have to check every element in the array, and there are N elements in total.

Average-case:

- Recall: $T_{\text{avg}}(N) = p \cdot \sum_{k=0}^{N-1} (1-p)^k (3k+5) + (1-p)^N (3N+4)$.
- Can show that if $0 < q < 1$, then $\sum_{k=0}^{N-1} q^k \cdot k = O(N)$.
- Applying previous rules gives $T_{\text{avg}}(N) = O(N)$. Still linear.

Space complexity

- So far we have been considering time complexity $\sim$ how much time it takes for an algorithm to run.
- Another notion of complexity is **space complexity** $\sim$ how much memory space an algorithm occupies during its running.
- Less commonly studied than time complexity. When we just say “complexity”, we mean “time complexity”.

Definition

The **space complexity** of an algorithm measures the amount of *auxiliary* memory space required for an execution of the algorithm.

- Here, “auxiliary” space means that we don’t include the input of the algorithm, since that is given as part of the problem.
- Instead, we measure the additional space required by our algorithm to solve the problem.

Worst- and average-case, and first example

- As with time complexity, we can define worst- and average-case space complexities as functions of the input size.
- Denote these $M_{\text{worst}}(N)$ (or just $M(N)$) and $M_{\text{avg}}(N)$.
- As with time complexity, estimate with Big-Oh notation.
- In this model, we consider that:
 - Simple data (e.g., numbers, booleans) occupy constant space;
 - Collections occupy a space equal to the sum of the spaces of their elements.

Example: The `LINEARSEARCH` procedure uses only simple variables n and i , so has (asymptotic) space complexity $M(N) = O(1)$ (constant space).

More complicated example: ISWELLBALANCED

Now let's analyse the complexity of the ISWELLBALANCED procedure.

- Here, the size of the input N is the length of the String `expr`.

Time complexity:

- No matter the input, we iterate over it.
- At each step of the loop, the number of operations is (bounded by a) constant.

Therefore, *linear* complexity $O(N)$ (worst- and average-cases).

General rule

If there are k loops $L_1, \dots, L_k$ nested inside each other such that, for each i , the number of repetitions of L_i is bounded above by a_i , and the loop L_i has otherwise only at most a constant number of operations, then we have $T(N) = O(a_1 \cdot a_2 \dots a_k)$.

More complicated example: ISWELLBALANCED

Space complexity:

- Intuition: the space complexity is the maximum size of the stack used in the procedure, since everything else occupies constant space.
- Worst-case: every character in `expr` is an opening delimiter, so the max stack size is the length of the String N , linear complexity.
- Average-case?
 - Model: input is a sequence of i.i.d. characters, uniformly chosen at random from some given character set (e.g. ASCII).
 - Key point: probability of opening and closing delimiters are equal (and fixed positive value).
 - The size of the stack is a balanced random walk.
 - Can show that the average size is $\sim c \cdot \sqrt{N}$, so $M_{\text{avg}}(N) = O(\sqrt{N})$.

Summary

- Algorithms: definition.
- Algorithm representation: pseudocode, flowcharts.
- Algorithm analysis: space/time complexity and Big-Oh estimates.